CMPT 295

Unit – Code Optimization

Lecture 36 – Exploiting Instruction-Level Parallelism – Loop unrolling

Last Lecture

- s/w developer-generated optimization:
  - Compiler-generated ***optimization strategies***
  - Memory reference (access) reduction
  - Branching minimization
- Also, s/w developers can help the compiler optimizing by transforming programs in such a way that the *optimization blockers* (function calls and memory aliasing) are removed

Today's Menu

- Code optimization
  - Examples of code optimization
    - Done by compilers
    - Done by software developers
  - More optimization strategies - Based on microprocessor architecture
    - Instruction parallelism
      - Loop unrolling
        - Re-association
        - Accumulators

During our unit on ISA (textbook Chapter 4)…

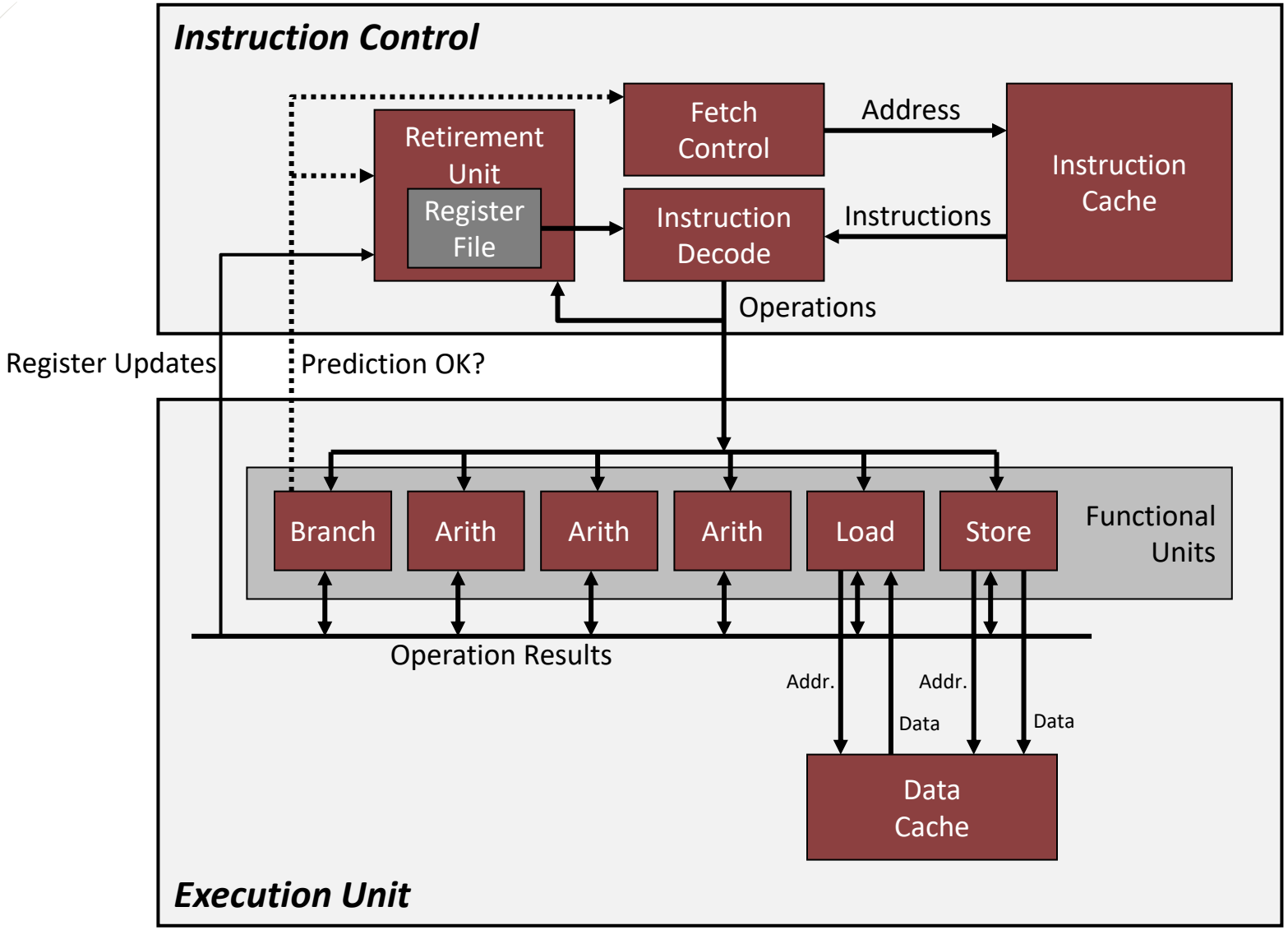
- … we were working with a *simple* ***in-order pipelined execution microprocessor*** *(Model 3 - Pipelined execution of machine code instructions)*
  - Datapath of this microprocessor is designed with stages forming a ***pipeline***
  - Microprocessor that executes machine code instructions ***in–order*** (as they are ordered in the executable)
  - ***1 machine code instruction executed per clock cycle (tick)***
- In this unit, we shall consider the type of microprocessors used in our ***target machine***
  - ***Superscalar microprocessor***
    - Most modern microprocessors are ***superscalar***
    - Example: Intel - since Pentium (1993)

In plain English ☺
This means …

Superscalar microprocessor

- Nowadays, microprocessors can execute ***multiple instructions per clock cycle***
  - ***Instruction-level parallelism***
- How doe ***Instruction-level parallelism*** work?
  - Datapath of a ***superscalar microprocessor*** composed of multiple ***functional units*** (see next slide)

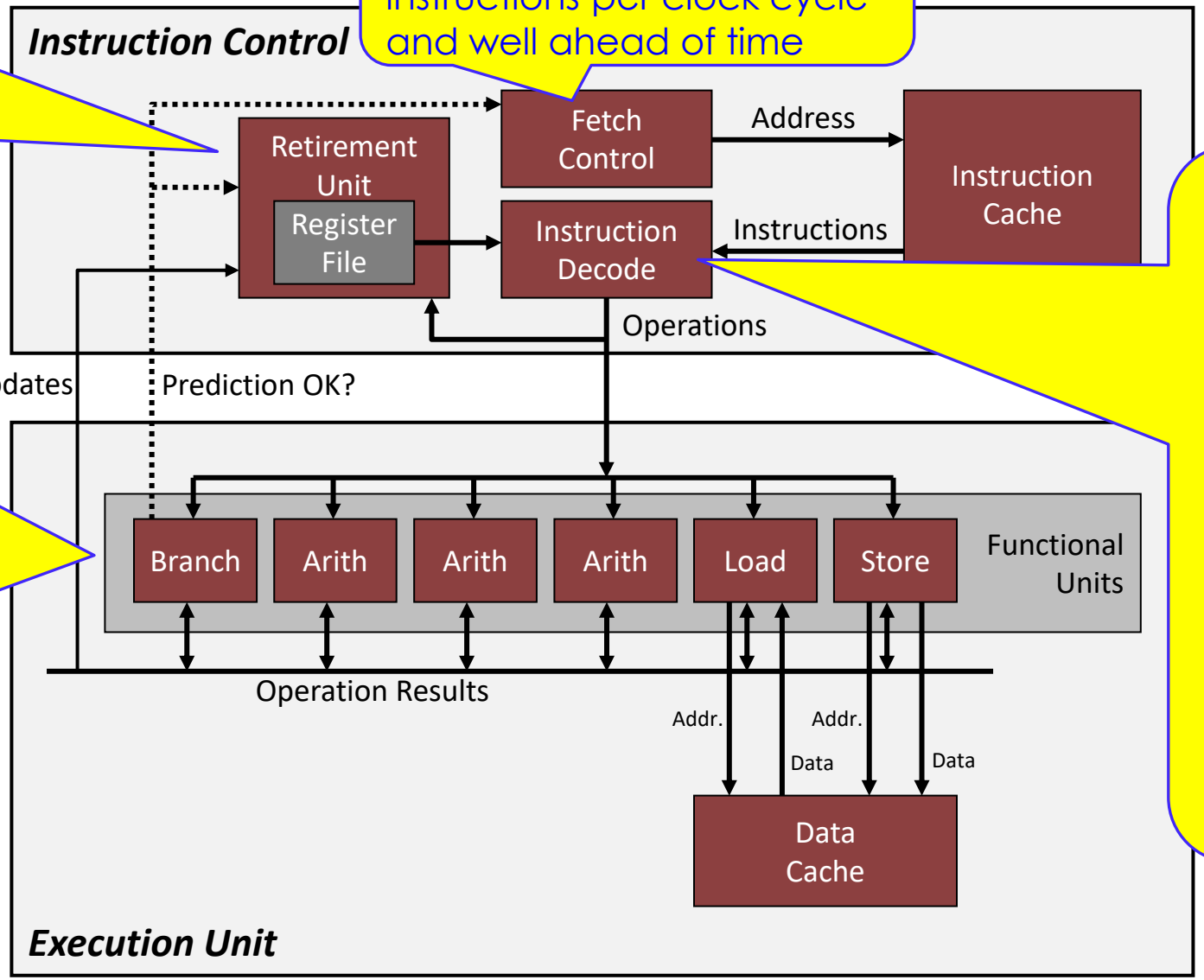
Example of ***superscalar microprocessor*** design (datapath)



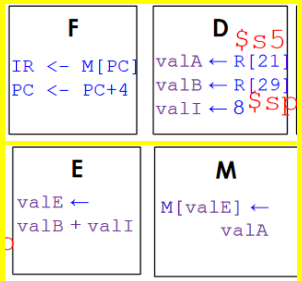
Example of ***superscalar microprocessor*** design (datapath)

1. Fetch Control Unit fetches many machine code instructions per clock cycle and well ahead of time

4. Retirement Unit puts everything back in order giving the impression that the machine code instructions have been executed sequentially as expected.



2. Instruction Decode Unit breaks down each machine code instruction into several micro-operations. Think of micro-operations as:



3. These micro-operations are queued to be executed by one of **functional units**.

To take advantage of ***instruction-level parallelism***, and to avoid hazards, **IDU** queues these micro-operations ***out of order*.**

Out of Order Execution (OoOE)

Because hazards negatively impact throughput!

- Microprocessors rearrange execution order of instructions to avoid hazards and to maximize the use of its ***functional units*** (hence maximize ***instruction-level parallelism***)
  - ***Out of Order Execution***
- Micro-operations may be executed ***out of order*** i.e., in a different order than the order of their associated machine code instructions, but the overall result is the same as the result expected by C s/w developer
  - Remember Slide 12 of Lecture 24:

Instruction Set Architecture (ISA) cont'd

4. Model of computation
   - Microprocessor executes our C program in such a way that it produces the expected result
     - We get the illusion that the microprocessor executes each C statement sequentially

s/w developer-generated optimization

- Software developer can restructure code to take advantage of ***Instruction level parallelism***
  - Loop unrolling
    - Re-associating
    - Accumulating

Strategy: Loop unrolling
But first let's set up the scene!

- First, let's start with "loop rolled"

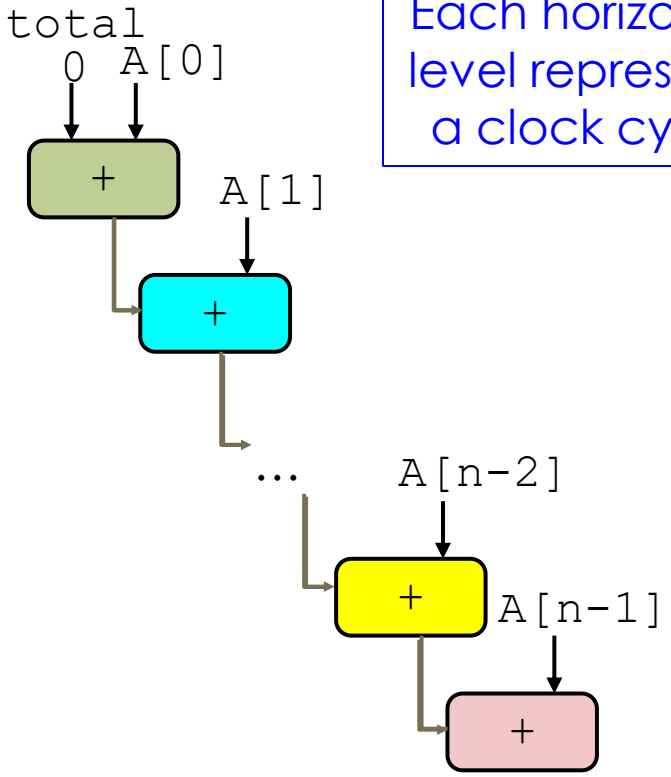
```
int sum_rolled(int *A, int n) {
    int total = 0;
    int i;

    for (i = 0; i < n; i++) {
        total += A[i];
    }

    return total;
}
```

Average of 1708 clock cycles

```
loop:
  # total += A[i]
  addl   (%rdi), %eax
  addq   $4, %rdi
  cmpq   %rdx, %rdi
  jne    loop
```



Each horizontal level represents a clock cycle

- Data dependency:
  - Each `total` depends on the `total` of previous iteration
  - Therefore, each `total` must be computed in sequence

Strategy: 2x1 loop unrolling

- Idea: Restructure code to allow multiple additions in parallel since we have a few Arith functional units

Average of 1454 clock cycles

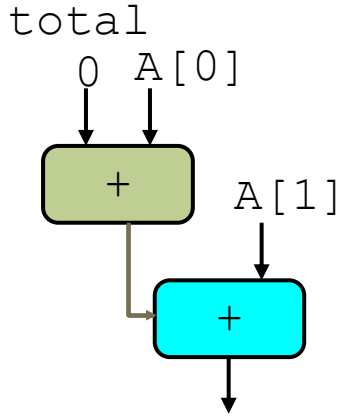
```
loop:
  # total += A[i]
  addl    (%rdx), %eax
  addq    $8, %rdx
  # total += A[i+1]
  addl    -4(%rdx), %eax
  cmpq    %rcx, %rdx
  jne     loop
```

```
int sum_2x1unrolled(int *A, int n) {
    int total = 0;
    int i;

    for (i = 0; i < n-1; i += 2) {
        total = (total + A[i]) + A[i+1];
    }
    for (; i < n; i++) {
        total += A[i];
    }

    return total;
}
```

Each horizontal level represents a clock cycle



One iteration requires two clock cycles:

Strategy: 2x1a loop unrolling (a -> association)

Average of 1352 clock cycles

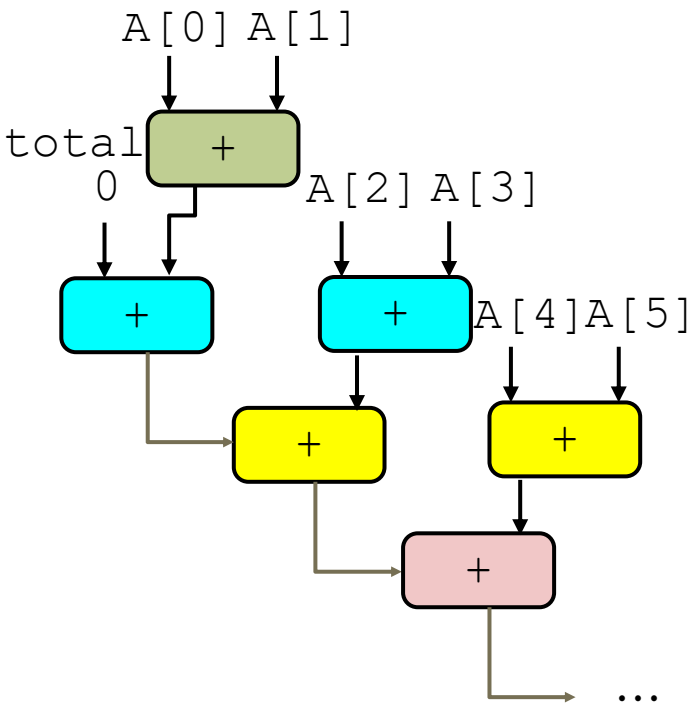
```
loop:
  movl   4(%rdx), %ecx
  # temp = A[i] + A[i+1]
  addl   (%rdx), %ecx
  addq   $8, %rdx
  # total += temp
  addl   %ecx, %eax
  cmpq   %r8, %rdx
  jne    loop
```

```
int sum_2x1aunrolled(int *A, int n) {
    int total = 0;
    int i;

    for (i = 0; i < n-1; i += 2) {
        total = total + (A[i] + A[i+1]);
    }
    for (; i < n; i++)
        total += A[i];

    return total;
}
```

Each horizontal level represents a clock cycle



Strategy: 2x2 loop unrolling + Accumulation

- Two sequences of independent additions

Average of 1297 clock cycles

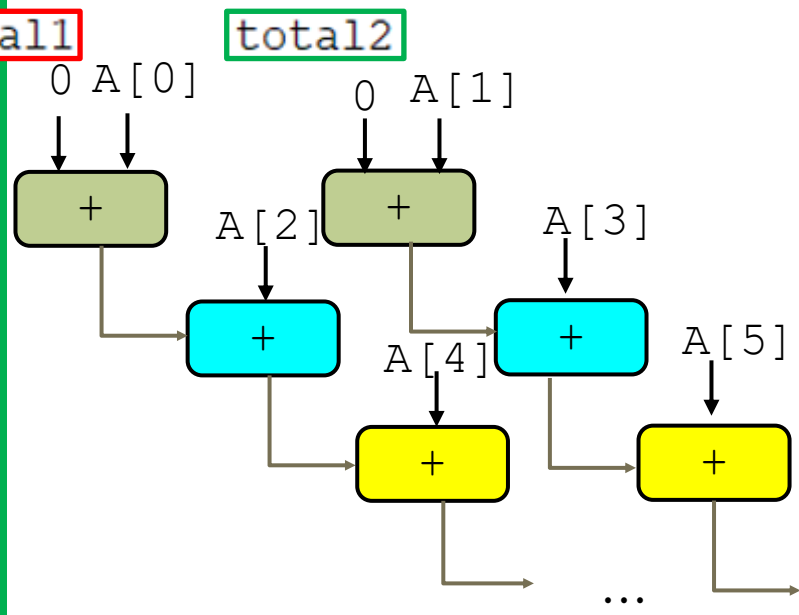
```
loop:
  # total1 += A[i]
  addl    (%rdx), %eax
  # total2 += A[i+1]
  addl    4(%rdx), %ecx
  addq    $8, %rdx
  cmpq    %r8, %rdx
  jne     loop
```

```
int sum_2x2unrolled(int *A, int n) {
    int total1 = 0;
    int total2 = 0;
    int granTotal = 0;
    int i;

    for (i = 0; i < n-1; i += 2) {
        total1 += A[i];
        total2 += A[i+1];
    }
    granTotal = total1 + total2;
    for (; i < n; i++)
        granTotal += A[i];

    return granTotal;
}
```

Each horizontal level represents a clock cycle



To summarize the unrolling strategy: k x c loop unrolling

- k -> the number of ***loop unrolling***, i.e., # of array elements we are dealing with within one iteration of the loop
- c -> the number of ***accumulators*** used within the loop
  - Like **`total1`** and **`total2`**

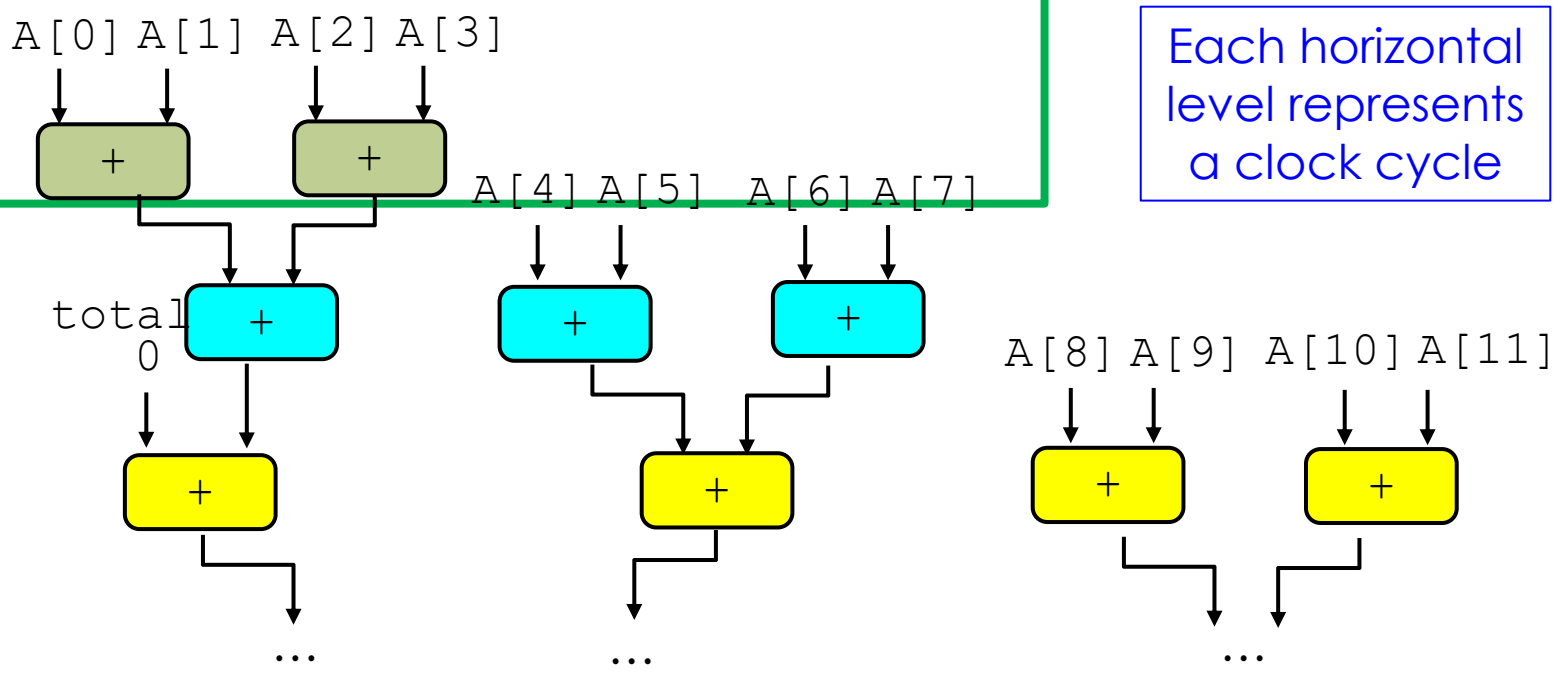
Average of 1421 clock cycles

Example of 4 x 1 loop unrolling

```
int sum_4x1aunrolled(int *A, int n) {
    int total = 0;
    int i;

    for (i = 0; i < n-3; i += 4) {
        total = total + ((A[i] + A[i+1]) + (A[i+2] + A[i+3]));
    }
    for (; i < n; i++)
        total += A[i];

    return total;
}
```



Each horizontal level represents a clock cycle

Example of 4 x 4 loop unrolling with accumulators

Average of 1129 clock cycles

```
int sum_4x4unrolled(int *A, int n) {
    int total1 = 0;
    int total2 = 0;
    int total3 = 0;
    int total4 = 0;
    int granTotal = 0;
    int i;

    for (i = 0; i < n-3; i += 4) {
        total1 += A[i];
        total2 += A[i+1];
        total3 += A[i+2];
        total4 += A[i+3];
    }
    granTotal = (total1 + total2) + (total3 + total4);

    for (; i < n; i++)
        granTotal += A[i];

    return granTotal;
}
```

Summary - 1

1. Superscalar microprocessor
   - Can execute multiple machine code instructions in one clock cycle since microprocessor has multiple ***functional units*** performing similar functions -> called ***instruction-level parallelism***
   - Rearrange execution order of machine code instructions to avoid hazards (negatively impact throughput) and to maximize the use of its ***functional units*** -> called ***out of order execution***
2. Optimization
   - Software developers can restructure code: ***loop unrolling*** **(re-associating + accumulating)**
   - **Effects**:
     - Reduce number of loop iterations (loop overhead)
     - Take advantage of ***instruction-level parallelism***
   - **Conclusion**: increase throughput!

Summary - 2

- However, let's keep in mind that …
- Our objective as software developers is to write programs that …
  - Solve the given problems
  - Are maintainable
  - Are robust
  - And are optimized … **only if *faster performance*** is required
- Why? Because optimization …
  - Increases code length, increases possibility of introducing bugs
  - Reduces readability, modularity …
  - Optimizing programs is ***time consuming*** and ***involves trade-offs***

Next Lecture

- Cache-friendly code optimization:
  - Locality
  - Memory hierarchy
  - Caching
- Cache memories
  - How does a cache work?
  - How is cache structured?
  - Examples
  - Cache performance analysis
  - Another example of cache -> conflict miss
- Memory mountain